

DAY 10 — Multiple JOINS in SQL

Today you will learn how to combine three or more tables in one query.

This is extremely important for Data Engineering and GenAI/RAG systems because real applications rarely store everything in one table.

For example:

users

↓

documents

↓

document_chunks

↓

embeddings

To answer questions such as:

“Show all documents uploaded by each user, along with their chunks and embedding status.”

you need multiple joins.

1. What Is a Multiple JOIN?

A multiple JOIN query combines more than two tables.

General syntax

```
SELECT
columns

FROM table1
JOIN table2
    ON table1.key = table2.foreign_key
JOIN table3
    ON table2.key = table3.foreign_key;
```

Example:

```
SELECT
users.name,

    orders.order_id,
    order_items.product_name
FROM users
JOIN orders
```

```
ON users.user_id = orders.user_id
JOIN order_items
ON orders.order_id = order_items.order_id;
```

The database processes the relationships step by step:

users

```
JOIN orders
JOIN order_items
```

2. Realistic RAG Database Schema

We will use these tables throughout today's lesson.

users

user_id	name	email
1	Om	om@example.com
2	Priya	priya@example.com
3	Rahul	rahul@example.com

documents

document_id	user_id	title	status
101	1	Python Guide	indexed
102	1	SQL Guide	processing
103	2	RAG Architecture	indexed
104	3	Docker Notes	failed

document_chunks

chunk_id	document_id	chunk_number	content
1001	101	1	Python introduction
1002	101	2	Python variables
1003	102	1	SQL SELECT
1004	103	1	RAG overview
1005	103	2	Vector search

embeddings

embedding_id	chunk_id	model_name
5001	1001	text-embedding-3-small
5002	1002	text-embedding-3-small
5003	1003	text-embedding-3-small
5004	1004	text-embedding-3-small

Notice:

- One user can have many documents.
- One document can have many chunks.
- One chunk may have one embedding.
- A document may exist without chunks.
- A chunk may exist without an embedding.

This is a typical relational structure.

3. Understanding the Relationship Chain

The relationship chain is:

users.user_id

↓

documents.user_id

documents.document_id

↓

document_chunks.document_id

document_chunks.chunk_id

↓

embeddings.chunk_id

The joins are:

users

JOIN documents

ON users.user_id = documents.user_id

JOIN document_chunks

ON documents.document_id = document_chunks.document_id

JOIN embeddings

ON document_chunks.chunk_id = embeddings.chunk_id

The important rule is:

Each JOIN should connect two tables through a logically related key.

4. Example: Users and Their Documents

Start with two tables.

```
SELECT
u.user_id,

u.name,

d.document_id,

d.title

FROM users AS u
JOIN documents AS d
  ON u.user_id = d.user_id;
```

This produces one row per matching user-document relationship.

Example result:

user_id	name	document_id	title
1	Om	101	Python Guide
1	Om	102	SQL Guide
2	Priya	103	RAG Architecture
3	Rahul	104	Docker Notes

Om appears twice because Om owns two documents.

This is not a duplicate caused by SQL. It represents the actual one-to-many relationship.

5. Adding the Third Table

Now add document_chunks.

```
SELECT

u.name,

d.title,

c.chunk_id,

c.chunk_number

FROM users AS u
JOIN documents AS d
  ON u.user_id = d.user_id
```